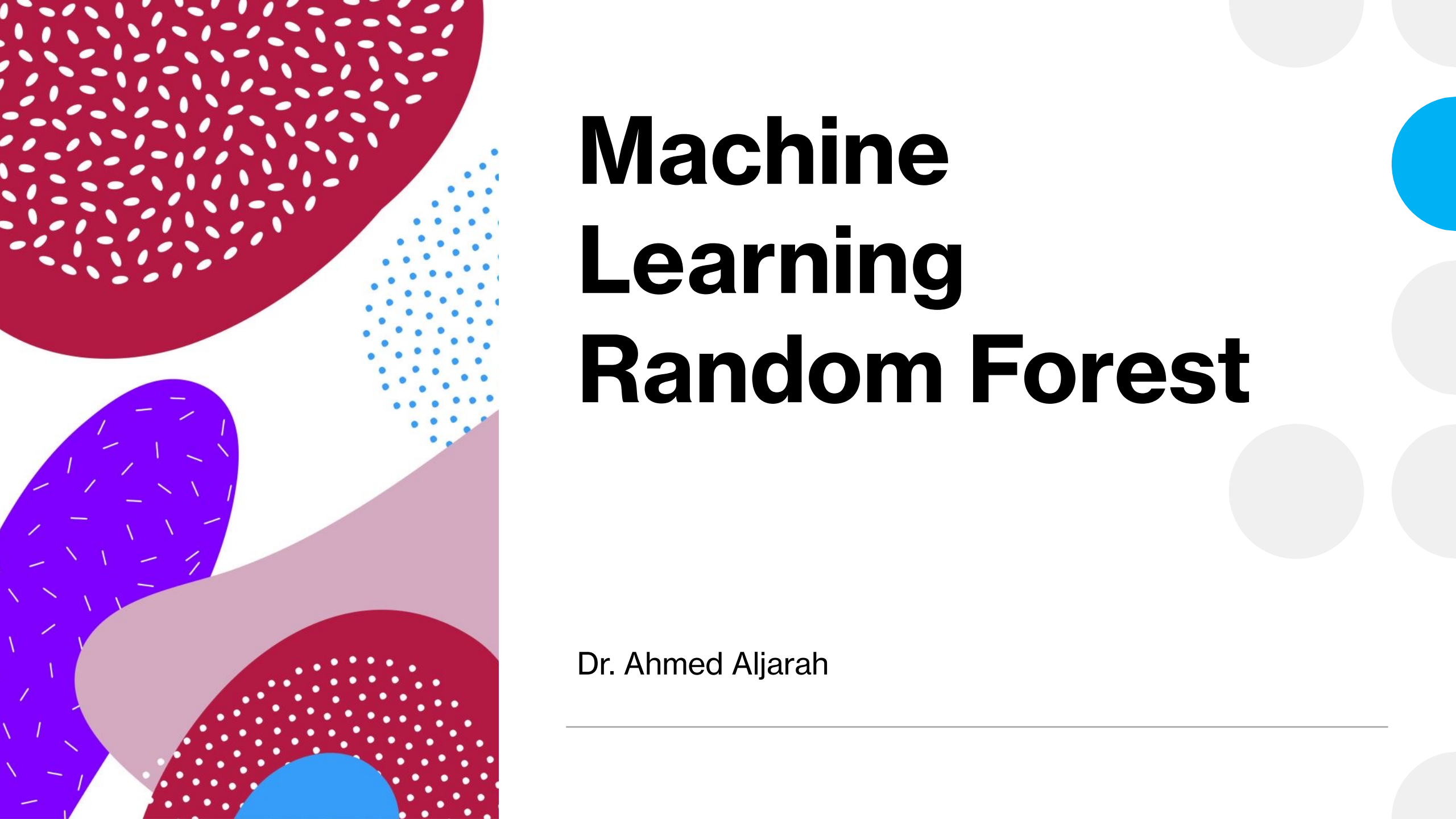


The background features several abstract shapes and patterns. On the left, there is a large red shape with a white dotted pattern, a blue shape with a white dashed pattern, and a purple shape with a white dotted pattern. On the right, there are several light gray circles and one bright blue circle. The title text is positioned in the upper right area of the slide.

Machine Learning Random Forest

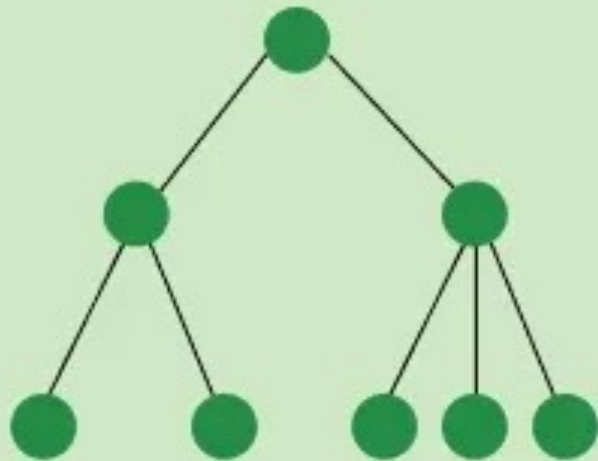
Dr. Ahmed Aljarah

Random forest

- Random Forest is a machine learning algorithm that uses multiple decision trees instead of one.
 - Each tree is trained on different data samples and random features.
 - For classification, it uses majority voting.
 - For regression, it averages the predictions.
 - It helps reduce overfitting compared to a single decision tree.
 - Works well with complex or noisy data.
 - Produces more accurate and stable results.
-

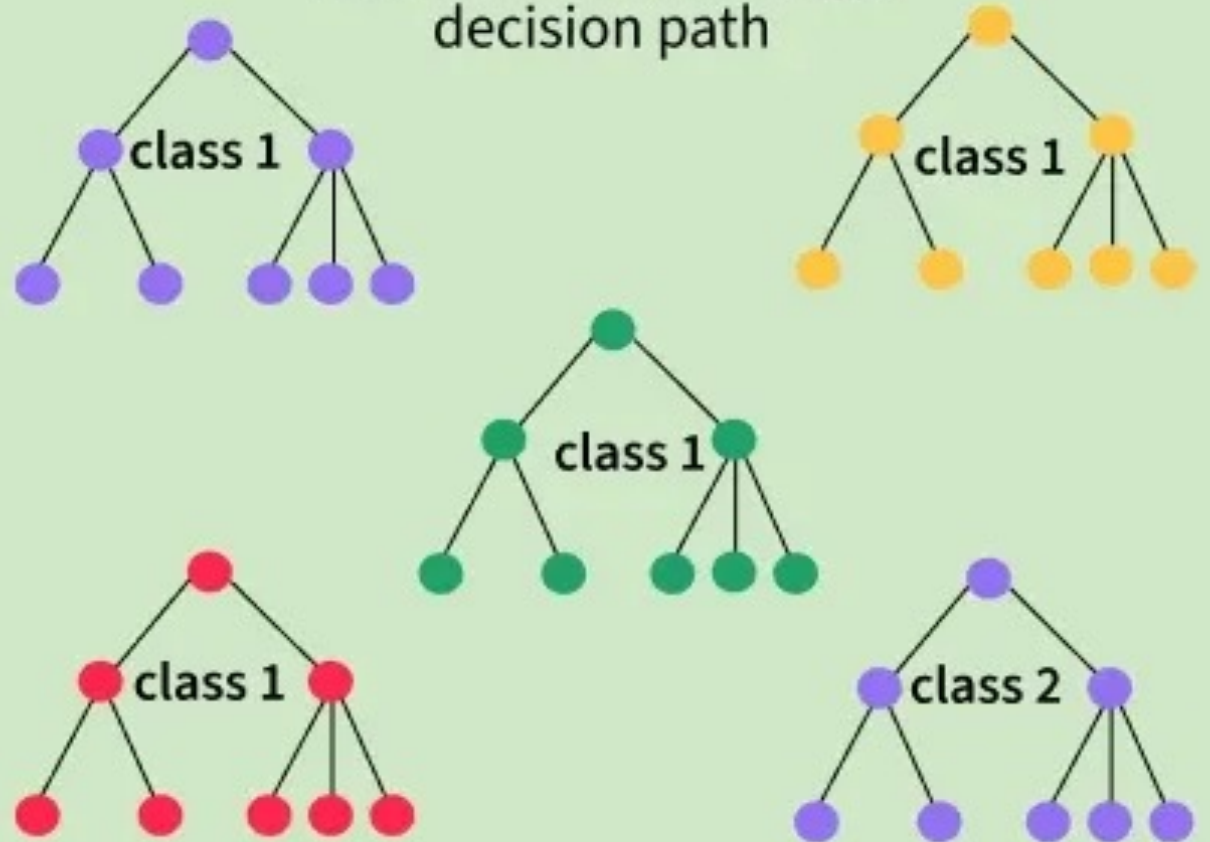
Single Decision Tree

Ensemble of trees for more accurate and robust prediction



Random Forest

Prediction from a single decision path



Random forest advantages

- No overfitting: uses multiple trees reduce the risk of overfitting.
 - High accuracy: runs efficiently on large database.
 - Estimates missing data: random forest can maintain accuracy when a large proportion of data is missing.
-

Random forest algorithm

- Builds many decision trees using different random samples of the data.
 - Selects a random subset of features for each split in a tree.
 - Each tree makes its own prediction.
 - Combines results:
 - Classification: majority voting
 - Regression: average of predictions
 - Reduces overfitting and improves accuracy by combining multiple trees.
-

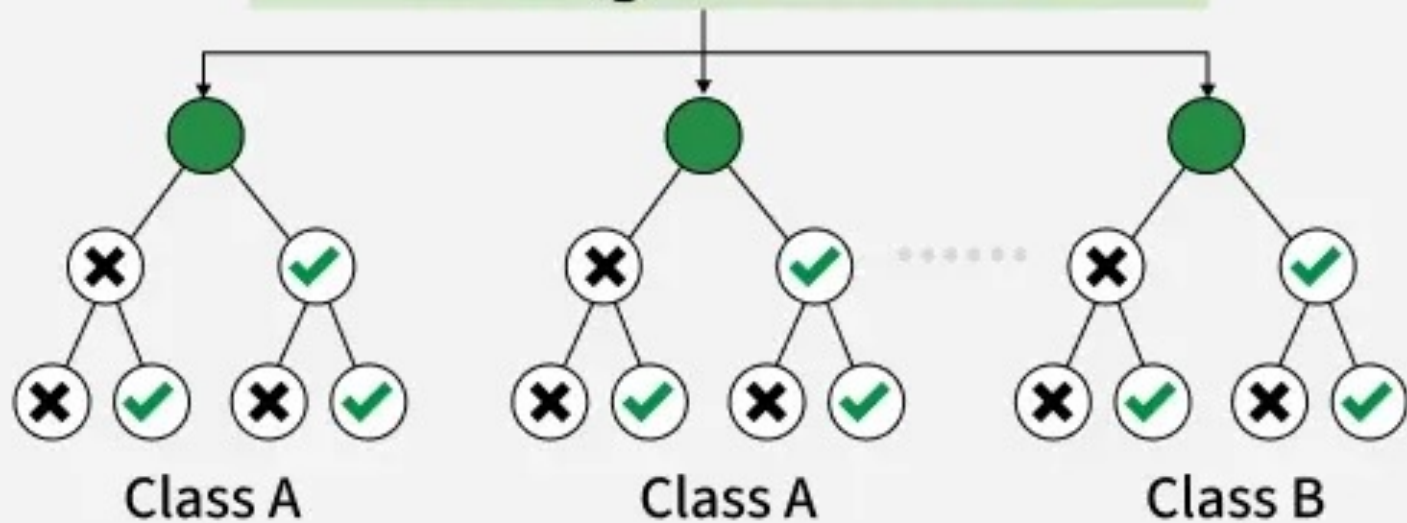
How Random forest works

- Each tree makes independent predictions.
 - Uses random samples and features for diversity.
 - Requires enough data to learn different patterns.
 - Combining multiple predictions improves overall accuracy.
-



Training Data Instance

Model
Training



Decision
Trees

Model
Testing

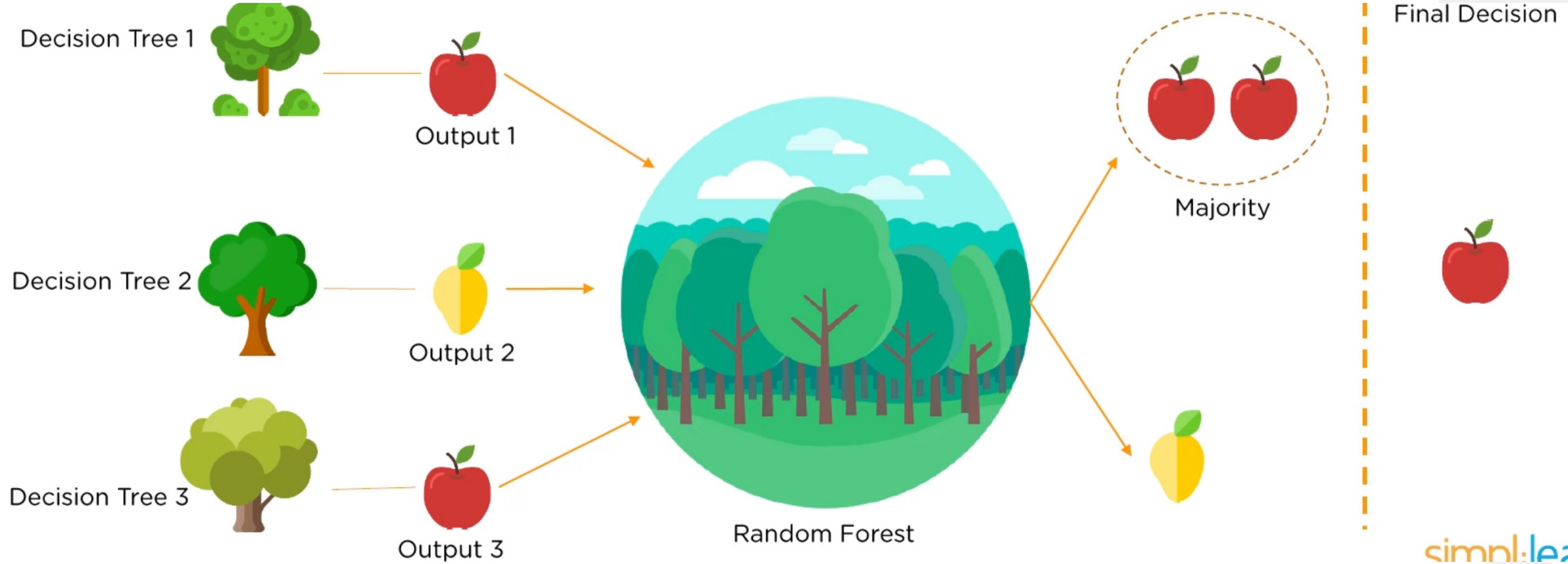
Bagging (Voting Majority)

Prediction Output

Class A

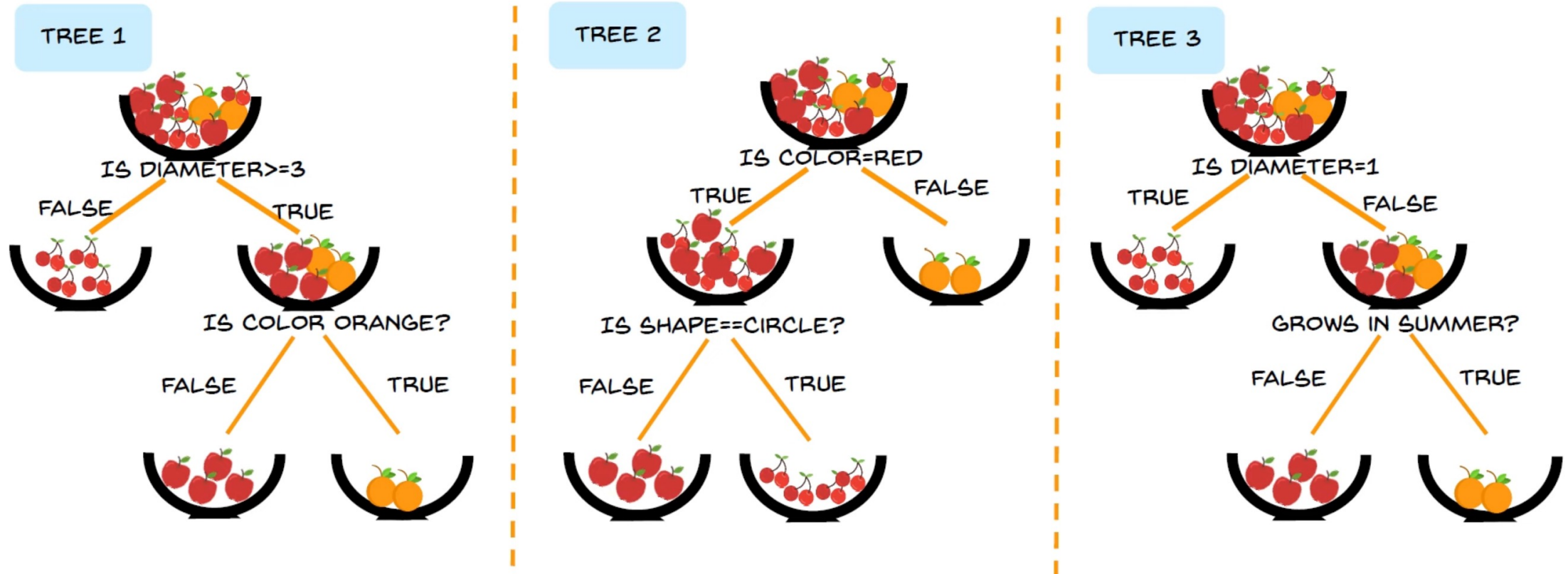
Example 1:

Random forest classification model

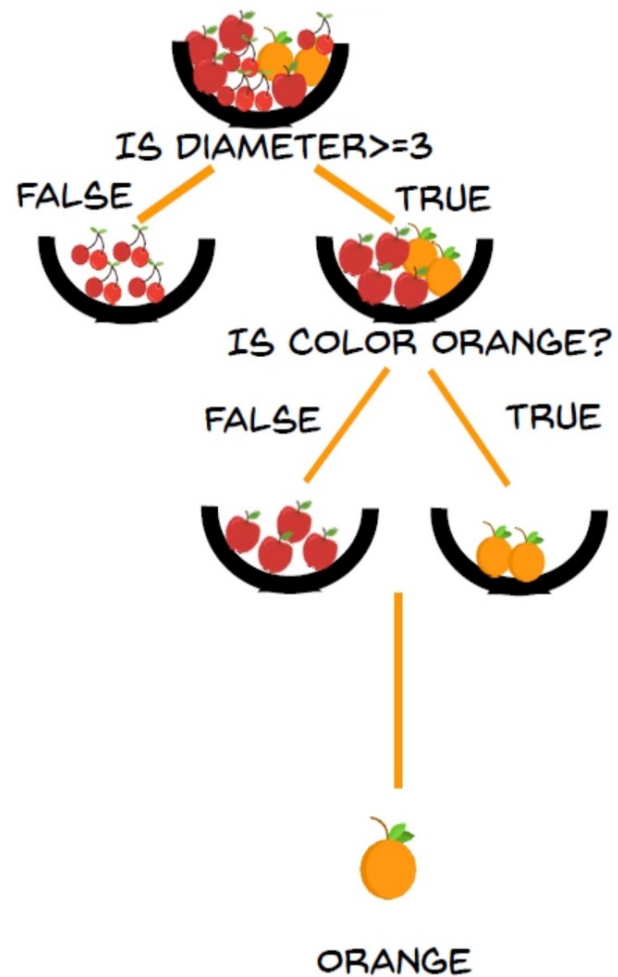


Example 2:

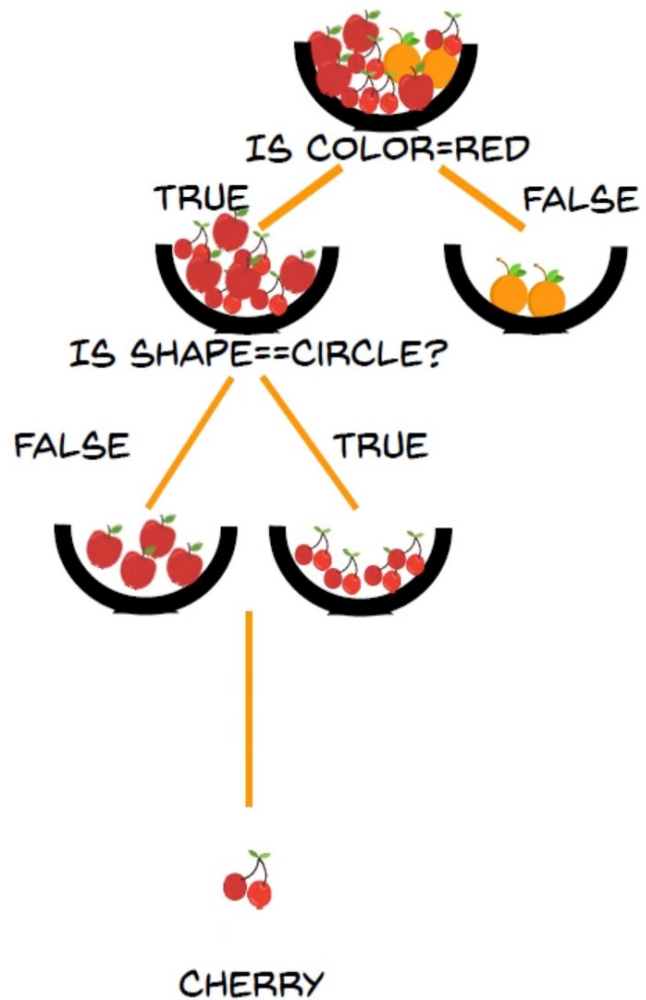
Random Forest Classification Model



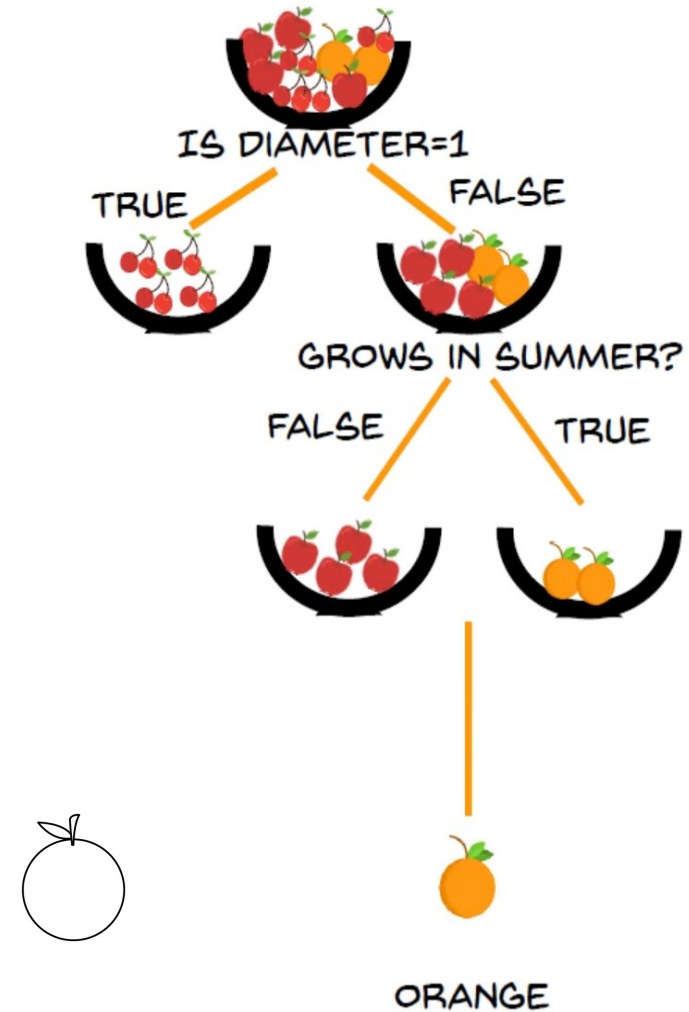
TREE 1



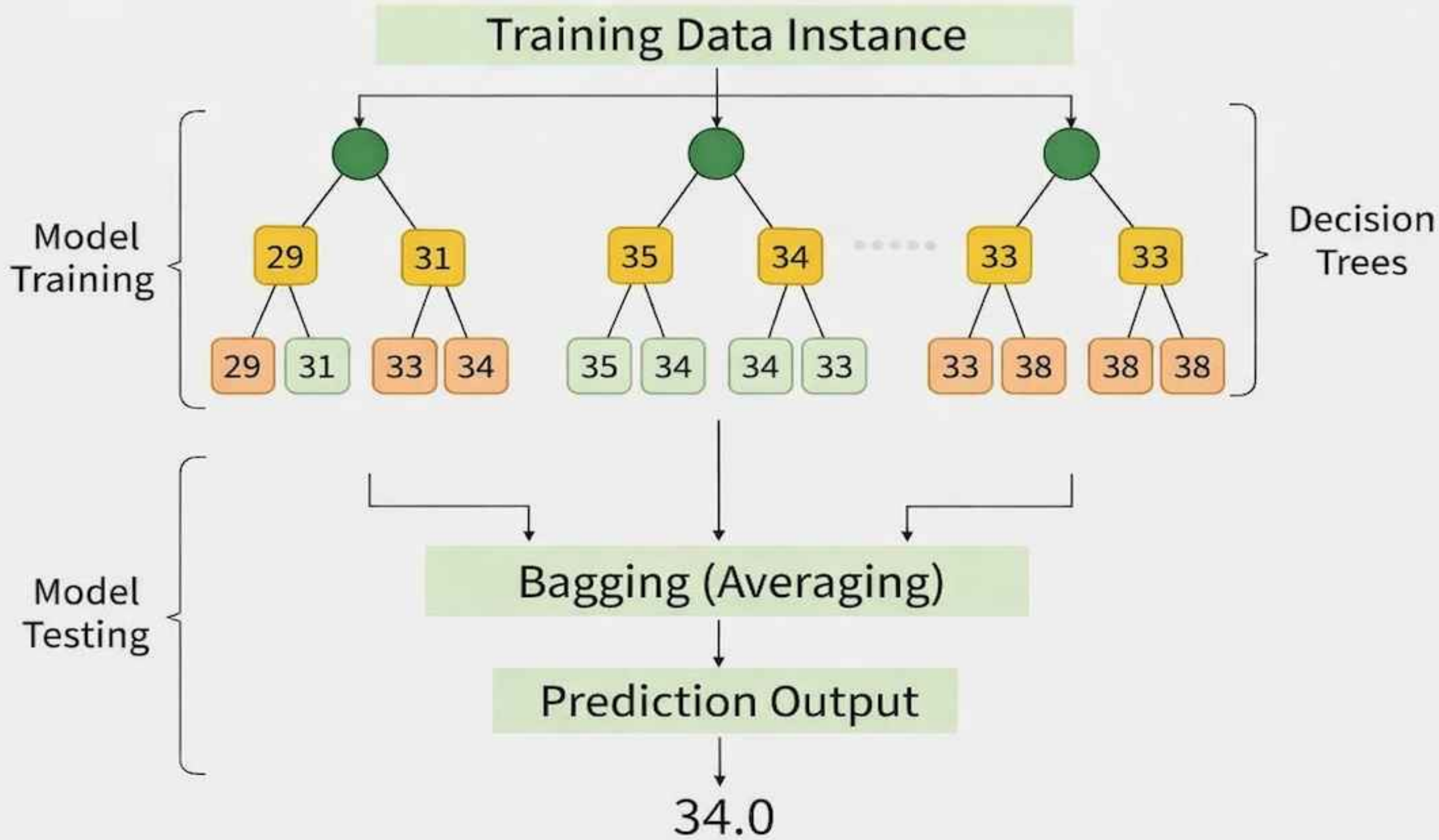
TREE 2



TREE 3



Diameter = 3
Grows in summer



Random forest builds trees by:

1

Using bootstrap dataset

2

Considering a random subset of variables at each step

Bootstrap sampling

- Method used to create datasets for bagging
 - It creates multiple training datasets from the original data
 - Each dataset is built by randomly selecting samples
 - Randomly selects data with replacement
 - Randomly selects data
 - With replacement, after picking a sample, put it back before picking again
 - Some data points may appear multiple times
 - Some data points may be left out
 - Creates slightly different datasets for each model
-

Bootstrap example:

Original dataset

Chest Pain	Good Blood circ	Block Arteries	Weight	Hard Disease
NO	NO	NO	125	NO
YES	YES	YES	180	YES
YES	YES	NO	210	NO
YES	NO	YES	167	YES

Bootstrapped dataset1

Chest Pain	Good Blood circ	Block Arteries	Weight	Hard Disease
YES	YES	YES	180	YES
NO	NO	NO	125	NO
YES	NO	YES	167	YES
YES	YES	NO	210	NO

Bootstrapped dataset 2

Chest Pain	Good Blood circ	Block Arteries	Weight	Hard Disease
YES	YES	YES	180	YES
NO	NO	NO	125	NO
YES	NO	YES	167	YES
YES	YES	YES	180	YES

Bootstrapped dataset 3

Chest Pain	Good Blood circ	Block Arteries	Weight	Hard Disease
YES	YES	YES	180	YES
YES	NO	YES	167	YES
YES	NO	YES	167	YES
YES	YES	NO	210	NO

Random subset of variables

First tree

Chest Pain	Weight	Hard Disease
YES	180	YES
NO	125	NO
YES	167	YES
YES	210	NO

Second tree

Chest Pain	Good Blood circ	Hard Disease
YES	NO	YES
YES	YES	NO
NO	NO	NO
YES	YES	YES

Third tree

Block Arteries	Weight	Hard Disease
YES	180	YES
NO	125	NO
YES	167	YES
NO	210	NO

Example of implementation of random forest, classification

- The iris dataset is used for this example
- It is a famous and simple dataset included in sklearn.datasets that is widely used for learning and testing machine learning models.

```
#Import libraries
from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassifier
import pandas as pd
import numpy as np
#Load iris dataset
iris = load_iris()
print(iris)
```

```
#create dataframe from the iris dataset
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df.head(10)

#Add the target to the dataframe
df["species"] = pd.Categorical.from_codes(iris.target,iris.target_names)
df.head()

#Determine the independent variables and dependent variable
x = df.values[:, : 4]
y = df.values[:,4]
print(y)

#Split dataset to train and test part
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test = train_test_split(x,y,test_size= 0.2,random_state = 2)
```

#Create the model

```
RF = RandomForestClassifier(n_estimators=100,random_state = 2)
RF.fit(x_train,y_train)
```

#predict the output of the test data

```
y_predict = RF.predict(x_test)
print(y_predict)
```

#Evaluate the model

```
from sklearn.metrics import accuracy_score, classification_report
accuracy_score(y_test, y_predict)
print("\nClassification Report:\n")
print(classification_report(y_test, y_predict))
```

Example of implementation of random forest, classification

```
#import libraries
import pandas as pd
import numpy as np
from sklearn.datasets import fetch_california_housing

#Load dataset
housing = fetch_california_housing()
print(housing)

#create Dataframe
df = pd.DataFrame(housing.data, columns= housing.feature_names )
df["MedHouseVal"] = housing.target
df.head()
```

```
#Determine the independent variables and dependent variable
x = df.values[:, :8]
y = df.values[:, 8]
print(y)

#Split dataset to train and test parts
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test=
train_test_split(x,y,test_size=0.2,random_state=42)

#Create the model
from sklearn.ensemble import RandomForestRegressor
RFR = RandomForestRegressor(n_estimators=100,random_state=42)
RFR.fit (x_train,y_train)

#Predict output
y_predict = RFR.predict(x_test)
print(y_predict)

# Evaluate the model
from sklearn.metrics import r2_score, mean_squared_error
rmse=np.sqrt(mean_squared_error(y_test,y_predict))
print("RMSE:", rmse)



---


print("R2_score: ",r2_score(y_test,y_predict))
```